Knowledge Organiser — 2.1 Elements of Computational Thinking

OCR A Level Computer Science (H446) — Component 02 — Spec ref 2.1.1–2.1.5

> Examined by **application to an unseen scenario** (game, booking system, robot, website), not by reciting definitions.

The five thinking skills

Skill (spec ref)	Definition	Apply-it cue
Thinking abstractly (2.1.1)	Removing/hiding unnecessary detail so only information relevant to the problem remains.	Name the detail you *keep* vs *discard* — and justify why the discarded detail is irrelevant *to this problem*.
Thinking ahead (2.1.2)	Planning before coding: inputs/outputs, preconditions, caching, reusable components.	List actual inputs/outputs, the precondition that must hold first, what to cache, and which modules to reuse.
Thinking procedurally (2.1.3)	Breaking a problem into an ordered sequence of steps and sub-procedures (decomposition).	Name scenario sub-procedures and state the fixed order (which steps depend on earlier outputs).
Thinking logically (2.1.4)	Identifying decision points where flow of control branches on a condition.	Give the exact Boolean condition (`IF x >= y`) and the outcome of *both* branches.
Thinking concurrently (2.1.5)	Spotting parts that can run at the same time (parallel/interleaved).	Identify *independent* tasks (no data dependency), then weigh benefit vs trade-off.

2.1.2 Thinking ahead — the four parts

Element	Key point
Inputs / Outputs	Data *in* vs results *out*. "Validate data" is a **process**, not an input/output.
Preconditions	Condition that must be true *before* a routine runs (e.g. list sorted before binary search).
Caching	Store expensive/frequent results in fast storage for reuse. **Benefit:** speed, less recompute/traffic. **Trade-off:** extra storage; data can go **stale** → needs invalidation.
Reusable components	Self-contained, generalised modules. **Benefits:** faster dev, fewer bugs (pre-tested), consistency, easier maintenance.

2.1.5 Thinking concurrently — benefits vs trade-offs

Benefits	Trade-offs
Faster completion / better throughput	Harder to design, program, debug
Better use of multi-core hardware	Risk of **race conditions** & **deadlock**
More responsive (UI free while task runs)	Results need synchronising/recombining
	Not always faster — single core = time-slicing, not true parallelism; overhead; some problems inherently sequential

Two types of abstraction (2.1.1)

- **Representational** — remove detail from *one specific* thing → simpler model of that problem (e.g. road network → graph of nodes + weighted edges).
- **Generalisation / categorisation** — group many things by *shared characteristics* → general reusable model (e.g. `Account` for savings + current).

⚠ Abstraction vs Decomposition (commonly confused)

- **Abstraction** = *removes/hides* unnecessary detail (simplify).
- **Decomposition** = *breaks* a problem into smaller sub-problems (split into sub-procedures).
- Different skills — never blur them.

Key terms

Term	Definition
Precondition	Must be true before a process runs correctly.
Stale data	Cached data out of date because the source changed.
Decision point	Place where flow branches on a condition (`IF` / `CASE` /loop test).
Flow of control	Order statements execute, shaped by decision points.
Race condition	Result depends on unpredictable timing of concurrent tasks sharing data.
Deadlock	Tasks each wait for a resource the other holds → none proceed.
Parallel processing	Multiple tasks literally simultaneous on multiple cores.

🎯 Top exam reminders

- **Apply, don't define.** Every point must name the scenario's actual inputs, conditions, sub-procedures — generic definitions score poorly on AO2/AO3.
- **Discuss = both sides.** Pair caching/concurrency *benefits* with their *trade-offs* (storage/stale data; race conditions/overhead).
- **Conditions must be exact Booleans** with both branches — "if enough seats" → `IF seatsRequested <= seatsAvailable`.

- **Check data dependencies** before claiming concurrency — dependent tasks must stay sequential; concurrency isn't automatically faster.
- **Watch the command word:** state/identify (AO1) = brief facts; describe/explain (AO2) = applied reasoning; discuss/evaluate/justify (AO3) = pros, cons + judgement.